

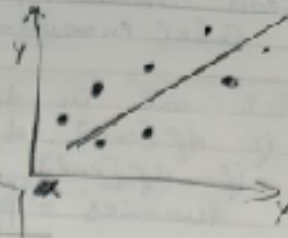
Linear Regression

- Linear family
- Baseline model
- Regression (continuous)
 - Players goals
 - Not injury
- Linear equation ($y = kx + b$)

Linear equation

x - input
 y - output
 β_0, β_1 - parameters
 ϵ - Error

$$y = \beta_0 + \beta_1 x + \epsilon$$



ma'lumotlarga eng mos keladigan
tugri chiziqni topadi va shu chiziq
yordamida yangi qiymatlarini prognoz
qiladi.

Kuchli tomonlari:

- Tez va sodda
- Kichik roq data lar bti ishlashda oson
- Base line model
- Kam chiqim

Kuchsiz tomonlari:

- Complex (non-linear) xolatlarini o'rganishda qiyin
- Yot elementlar da tez o'zgaruvchan
- Katta data lar da yaxshi ishlamaydi

df. head () - 1-5 ta qatorni ko'rsatadi

df. info () - datalardagi ma'lumot

df. isnull (). sum () - tushib qolgan qiymat

Model Selection

```
from sklearn.linear_model import  
LinearRegression
```

Training:

```
X = df.drop('Target', axis=1)  
y = df['Target']
```

```
from sklearn.model_selection import  
train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_  
split(X, y, test_size=0.2, rand  
random_state=42)
```

```
X_train.shape
```

```
X_test.shape
```

```
lr.fit(X_train, y_train)
```

Prediction:

```
y_pred = lr.predict(X_test)
```

```
y_pred[0]
```

Evaluation: (barakaydi)

- Classification

- Accuracy score, Classification Report

Regression

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- R^2 score (Coefficient of Determination)

from sklearn.metrics import mean

- absolute_error, mean_squared_error, r_score

mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

r2 = r2_score(y_test, y_pred)

Caption

```
from sklearn.preprocessing import  
Label Encoding
```

```
for col in df.columns:  
    if df[col].dtype == "object":  
        if df[col].nunique() <= 5:  
            dummies = pd.get_dummies(df[col],  
                                      prefix = col, dtype = int)  
            df = pd.concat([df.drop(columns =  
                                = col), dummies], axis = 1)  
        else:  
            df[col] = encoder.fit_transform(df[col])
```

```
from sklearn.preprocessing import  
MinMaxScaler
```

```
Scaler = MinMaxScaler()
```

```
num_col = df.select_dtypes(include =  
                             = ['int64', 'float64']).columns.drop('Target')  
num_col
```

```
df[num_col] = Scaler.fit_transform(  
df[num_col])
```